

65816 — compact reference

This is a **compact, 16-bit-focused** 65816 reference: the processor model needed to drive `+mos-a16` codegen (native mode, the M/X width flags, and how you move between 8- and 16-bit A / X / Y), followed by the full instruction set. Per-instruction 8-bit-only minutiae (cycle-by-cycle timing, emulation-mode quirks) are deliberately omitted — see the canonical sources for those.

It is **generated from the llvm-mos backend's own TableGen** (so it matches the assembler/codegen you actually run) and every opcode is **cross-checked against a canonical opcode matrix**; the result of that check is the companion [opcode audit](#).

Processor model

The 65816 powers up in **6502 emulation mode** ($E = 1$). The SNES `crt0` switches to **native mode** once, with `CLC`; `XCE` (exchange Carry into the E flag). In native mode two status-register bits set operand width independently:

Flag	Bit	0 ⇒	1 ⇒
M	5	16-bit accumulator / memory	8-bit accumulator / memory
X	4	16-bit index (X, Y)	8-bit index (X, Y)

`+mos-a16` runs with **M = 0** (16-bit accumulator). Width is changed with **REP** (reset P bits) and **SEP** (set P bits):

Want	Instruction
16-bit A	<code>REP #\$20</code>
16-bit X/Y	<code>REP #\$10</code>
16-bit A + X/Y	<code>REP #\$30</code>
8-bit A	<code>SEP #\$20</code>
8-bit X/Y	<code>SEP #\$10</code>

The width gotcha: an immediate operand's size (and how many bytes the instruction occupies) follows the M/X flag *as seen by the assembler at that point*. `LDA #$1234` is a 3-byte instruction when M = 0 but `LDA #$12` is 2 bytes when M = 1 — so REP/SEP placement and the assembler's tracking of M/X must agree, or the operand bytes shear. Accumulator/memory instructions (ADC, AND, CMP, EOR, LDA, ORA, SBC, BIT, ...) follow **M**; index instructions (LDX, LDY, CPX, CPY) follow **X**. Setting X = 1 zeroes the high bytes of X and Y. `XBA` swaps the two halves of A; `TCD/TDC/TCS/TSC` move the full 16 bits regardless of M.

In the tables below, **Bytes** is the instruction length the backend encodes; for the M/X-width immediates it is the 8-bit form (add 1 when the governing flag selects 16-bit).

Instruction set

Generated from the backend; opcode, mnemonic and byte length are the backend's, addressing-mode notation is canonical. Sorted by mnemonic. (`i` implied, `A` accumulator, `#` immediate, `dp` direct page, `abs` absolute, `long` absolute-long, `(...)` / `[...]` indirect/indirect-long, `,X` / `,Y` / `,S` indexed, `rel` / `rlong` branch.)

Mnemonic	Mode	Opcode	Bytes
ADC	(dp,X)	\$61	2
ADC	sr,S	\$63	2
ADC	dp	\$65	2
ADC	[dp]	\$67	2
ADC	#	\$69	2
ADC	abs	\$6D	3
ADC	long	\$6F	4
ADC	(dp),Y	\$71	2
ADC	(dp)	\$72	2
ADC	(sr,S),Y	\$73	2
ADC	dp,X	\$75	2
ADC	[dp],Y	\$77	2

Mnemonic	Mode	Opcode	Bytes
ADC	abs,Y	\$79	3
ADC	abs,X	\$7D	3
ADC	long,X	\$7F	4
AND	(dp,X)	\$21	2
AND	sr,S	\$23	2
AND	dp	\$25	2
AND	[dp]	\$27	2
AND	#	\$29	2
AND	abs	\$2D	3
AND	long	\$2F	4
AND	(dp),Y	\$31	2
AND	(dp)	\$32	2
AND	(sr,S),Y	\$33	2
AND	dp,X	\$35	2
AND	[dp],Y	\$37	2
AND	abs,Y	\$39	3
AND	abs,X	\$3D	3
AND	long,X	\$3F	4
ASL	dp	\$06	2
ASL	A	\$0A	1
ASL	abs	\$0E	3
ASL	dp,X	\$16	2

Mnemonic	Mode	Opcode	Bytes
ASL	abs,X	\$1E	3
BCC	rel	\$90	2
BCS	rel	\$B0	2
BEQ	rel	\$F0	2
BIT	dp	\$24	2
BIT	abs	\$2C	3
BIT	dp,X	\$34	2
BIT	abs,X	\$3C	3
BIT	#	\$89	2
BMI	rel	\$30	2
BNE	rel	\$D0	2
BPL	rel	\$10	2
BRA	rel	\$80	2
BRK	i	\$00	1
BRL	rlong	\$82	3
BVC	rel	\$50	2
BVS	rel	\$70	2
CLC	i	\$18	1
CLD	i	\$D8	1
CLI	i	\$58	1
CLV	i	\$B8	1
CMP	(dp,X)	\$C1	2

Mnemonic	Mode	Opcode	Bytes
CMP	sr,S	\$C3	2
CMP	dp	\$C5	2
CMP	[dp]	\$C7	2
CMP	#	\$C9	2
CMP	abs	\$CD	3
CMP	long	\$CF	4
CMP	(dp),Y	\$D1	2
CMP	(dp)	\$D2	2
CMP	(sr,S),Y	\$D3	2
CMP	dp,X	\$D5	2
CMP	[dp],Y	\$D7	2
CMP	abs,Y	\$D9	3
CMP	abs,X	\$DD	3
CMP	long,X	\$DF	4
CPX	#	\$E0	2
CPX	dp	\$E4	2
CPX	abs	\$EC	3
CPY	#	\$C0	2
CPY	dp	\$C4	2
CPY	abs	\$CC	3
DEC	A	\$3A	1
DEC	dp	\$C6	2

Mnemonic	Mode	Opcode	Bytes
DEC	abs	\$CE	3
DEC	dp,X	\$D6	2
DEC	abs,X	\$DE	3
DEX	i	\$CA	1
DEY	i	\$88	1
EOR	(dp,X)	\$41	2
EOR	sr,S	\$43	2
EOR	dp	\$45	2
EOR	[dp]	\$47	2
EOR	#	\$49	2
EOR	abs	\$4D	3
EOR	long	\$4F	4
EOR	(dp),Y	\$51	2
EOR	(dp)	\$52	2
EOR	(sr,S),Y	\$53	2
EOR	dp,X	\$55	2
EOR	[dp],Y	\$57	2
EOR	abs,Y	\$59	3
EOR	abs,X	\$5D	3
EOR	long,X	\$5F	4
INC	A	\$1A	1
INC	dp	\$E6	2

Mnemonic	Mode	Opcode	Bytes
INC	abs	\$EE	3
INC	dp,X	\$F6	2
INC	abs,X	\$FE	3
INX	i	\$E8	1
INY	i	\$C8	1
JML	(abs)	\$DC	3
JMP	abs	\$4C	3
JMP	long	\$5C	4
JMP	(abs)	\$6C	3
JMP	(dp,X)	\$7C	3
JSL	long	\$22	4
JSR	abs	\$20	3
JSR	(abs,X)	\$FC	3
LDA	(dp,X)	\$A1	2
LDA	sr,S	\$A3	2
LDA	dp	\$A5	2
LDA	[dp]	\$A7	2
LDA	#	\$A9	2
LDA	abs	\$AD	3
LDA	long	\$AF	4
LDA	(dp),Y	\$B1	2
LDA	(dp)	\$B2	2

Mnemonic	Mode	Opcode	Bytes
LDA	(sr,S),Y	\$B3	2
LDA	dp,X	\$B5	2
LDA	[dp],Y	\$B7	2
LDA	abs,Y	\$B9	3
LDA	abs,X	\$BD	3
LDA	long,X	\$BF	4
LDX	#	\$A2	2
LDX	dp	\$A6	2
LDX	abs	\$AE	3
LDX	dp,Y	\$B6	2
LDX	abs,Y	\$BE	3
LDY	#	\$A0	2
LDY	dp	\$A4	2
LDY	abs	\$AC	3
LDY	dp,X	\$B4	2
LDY	abs,X	\$BC	3
LSR	dp	\$46	2
LSR	A	\$4A	1
LSR	abs	\$4E	3
LSR	dp,X	\$56	2
LSR	abs,X	\$5E	3
MVN	src,dst	\$54	3

Mnemonic	Mode	Opcode	Bytes
MVP	src,dst	\$44	3
NOP	i	\$EA	1
ORA	(dp,X)	\$01	2
ORA	sr,S	\$03	2
ORA	dp	\$05	2
ORA	[dp]	\$07	2
ORA	#	\$09	2
ORA	abs	\$0D	3
ORA	long	\$0F	4
ORA	(dp),Y	\$11	2
ORA	(dp)	\$12	2
ORA	(sr,S),Y	\$13	2
ORA	dp,X	\$15	2
ORA	[dp],Y	\$17	2
ORA	abs,Y	\$19	3
ORA	abs,X	\$1D	3
ORA	long,X	\$1F	4
PEA	abs	\$F4	3
PEI	(dp)	\$D4	2
PER	rlong	\$62	3
PHA	i	\$48	1
PHB	i	\$8B	1

Mnemonic	Mode	Opcode	Bytes
PHD	i	\$0B	1
PHK	i	\$4B	1
PHP	i	\$08	1
PHX	i	\$DA	1
PHY	i	\$5A	1
PLA	i	\$68	1
PLB	i	\$AB	1
PLD	i	\$2B	1
PLP	i	\$28	1
PLX	i	\$FA	1
PLY	i	\$7A	1
REP	#	\$C2	2
R0L	dp	\$26	2
R0L	A	\$2A	1
R0L	abs	\$2E	3
R0L	dp,X	\$36	2
R0L	abs,X	\$3E	3
R0R	dp	\$66	2
R0R	A	\$6A	1
R0R	abs	\$6E	3
R0R	dp,X	\$76	2
R0R	abs,X	\$7E	3

Mnemonic	Mode	Opcode	Bytes
RTI	i	\$40	1
RTL	i	\$6B	1
RTS	i	\$60	1
SBC	(dp,X)	\$E1	2
SBC	sr,S	\$E3	2
SBC	dp	\$E5	2
SBC	[dp]	\$E7	2
SBC	#	\$E9	2
SBC	abs	\$ED	3
SBC	long	\$EF	4
SBC	(dp),Y	\$F1	2
SBC	(dp)	\$F2	2
SBC	(sr,S),Y	\$F3	2
SBC	dp,X	\$F5	2
SBC	[dp],Y	\$F7	2
SBC	abs,Y	\$F9	3
SBC	abs,X	\$FD	3
SBC	long,X	\$FF	4
SEC	i	\$38	1
SED	i	\$F8	1
SEI	i	\$78	1
SEP	#	\$E2	2

Mnemonic	Mode	Opcode	Bytes
STA	(dp,X)	\$81	2
STA	sr,S	\$83	2
STA	dp	\$85	2
STA	[dp]	\$87	2
STA	abs	\$8D	3
STA	long	\$8F	4
STA	(dp),Y	\$91	2
STA	(dp)	\$92	2
STA	(sr,S),Y	\$93	2
STA	dp,X	\$95	2
STA	[dp],Y	\$97	2
STA	abs,Y	\$99	3
STA	abs,X	\$9D	3
STA	long,X	\$9F	4
STP	i	\$DB	1
STX	dp	\$86	2
STX	abs	\$8E	3
STX	dp,Y	\$96	2
STY	dp	\$84	2
STY	abs	\$8C	3
STY	dp,X	\$94	2
STZ	dp	\$64	2

Mnemonic	Mode	Opcode	Bytes
STZ	dp,X	\$74	2
STZ	abs	\$9C	3
STZ	abs,X	\$9E	3
TAX	i	\$AA	1
TAY	i	\$A8	1
TCD	i	\$5B	1
TCS	i	\$1B	1
TDC	i	\$7B	1
TRB	dp	\$14	2
TRB	abs	\$1C	3
TSB	dp	\$04	2
TSB	abs	\$0C	3
TSC	i	\$3B	1
TSX	i	\$BA	1
TXA	i	\$8A	1
TXS	i	\$9A	1
TXY	i	\$9B	1
TYA	i	\$98	1
TYX	i	\$BB	1
WAI	i	\$CB	1
XBA	i	\$EB	1
XCE	i	\$FB	1

65816 backend opcode audit

Audits the **llvm-mos backend's** 65816 instruction encodings (from its TableGen) against an independent **canonical opcode matrix** (CC0; see [SOURCES.md](#)). A mismatch is a candidate backend defect. Generated by `tools/gen-65816-ref.py`.

Summary

Result	Opcodes
✓ exact agreement (mnemonic + bytes)	253
≈ byte-count differs by the expected width/signature rule	1
~ mnemonic synonym (e.g. JMP-long ≡ JML)	0
⚠ unexplained byte-count mismatch	0
✗ canonical opcode absent from the backend	1
— reserved in canon (WDM etc.)	1

254 / 255 defined opcodes agree (exact, width-rule, or synonym). **See discrepancies below.**

Notable rows

Opcode	Canonical	Backend	Note
<code>\$00</code> ≈	<code>BRK s</code> / 2B	<code>BRK</code> Implied / 1B	M/X-width or signature byte (expected)
<code>\$02</code> ✗	<code>COP s</code> / 2B	(none)	backend has no matching mnemonic at this opcode

Backend instructions at canon-reserved opcodes

Opcode	Backend	Note
\$42	WDM	reserved in the datasheet matrix